

Reviewer Pack — Matroid Rank and Minimal Circuit Size for Boolean Functions

Entry 4d473e78494e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 03:07:14 UTC

Matroid Rank and Minimal Circuit Size for Boolean Functions

Entry ID: 4d473e78494e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 03:07:14 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Minimal Circuit Size (MCSP)

Statement:

For any boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal circuit size of f is at least the rank of the matroid formed by the row space of its truth table over $\text{GF}(2)$.

Rationale (proposer's reasoning):

The linear algebra structure of the truth table imposes constraints on the circuit complexity, suggesting that the rank of the matroid provides a lower bound on the minimal circuit size.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9131010fa8b587c9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: INCONCLUSIVE against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - matroid rank minimal circuit size boolean function - row space GF(2) circuit lower bounds matroid theory - boolean function complexity matroid rank circuit complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1112.2313v2>] QBF-Based Boolean Function Bi-Decomposition - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/2304.02770v2>] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [<http://arxiv.org/abs/1810.06267v2>] Small Space Stream Summary for Matroid Center - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for j in range(cols):
        i_max = rank
        for i in range(rank, rows):
            if abs(matrix[i][j]) > abs(matrix[i_max][j]):
                i_max = i
        if matrix[i_max][j] == 0:
            continue
        matrix[rank], matrix[i_max] = matrix[i_max], matrix[rank]
        for i in range(rows):
            if i != rank and matrix[i][j] != 0:
                factor = -matrix[i][j] / matrix[rank][j]
                for k in range(cols):
                    matrix[i][k] += factor * matrix[rank][k]
        rank += 1
    return rank

def matrix_multiplication(A, B):
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])
    C = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```

def generate_truth_table(n):
    return [[random.randint(0, 1) for _ in range(2**n)] for _ in range(2**n)]

def min_circuit_size(f):
    n = len(f)
    truth_table = generate_truth_table(n)
    A = []
    for row in truth_table:
        A.append([row[i] for i in range(n)])
    rank = gaussian_elimination(A)
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    f = [random.randint(0, 1) for _ in range(2**n)]
    truth_table = generate_truth_table(n)
    A = []
    for row in truth_table:
        A.append([row[i] for i in range(n)])
    rank = gaussian_elimination(A)
    min_size = min_circuit_size(f)
    conjecture_holds = rank <= min_size
    counterexample = "" if conjecture_holds else f"min_circuit_size={min_size}, matroid_rank={rank}"
    return {
        "metric_name": "min_circuit_size",
        "metric_value": min_size,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))**0.5
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={res[first_failing_seed]['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing conclusive results; cannot determine support fraction or counterexample. | next: Increase timeout duration and re-run test with optimized code to handle larger instances

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107928
2	propose	ollama_remote	qwen3:8b	0	0	144636
3	preregistration	ollama_remote	qwen3:8b	0	0	24290
4	novelty	ollama_remote	qwen3:8b	0	0	19273
5	novelty	ollama_remote	qwen3:8b	0	0	24928
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11501
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11380
8	judge	ollama_remote	qwen3:8b	0	0	18266

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 362203 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/4d473e78494e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4d473e78494e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4d473e78494e.tar.gz` (if generated)